Sketch to Image

Step 1: Creating the sketch images from the rgb images

Code

1. Environment & Setup

The script begins by mounting **Google Drive** to access your data and creating a destination folder for the output.

- **GPU Acceleration:** It checks for a CUDA-enabled GPU. By moving the image data to the GPU (.to(device)), the mathematical operations (convolutions) happen much faster than they would on a standard processor (CPU).
- **Preprocessing:** It defines a transform that resizes every image to **512x512** pixels and converts them into a **Tensor** (a multi-dimensional matrix that PyTorch can manipulate).

---

2. The "Sketch" Logic (Sobel Edge Detection)

The core of the "sketch" effect is created using a classic computer vision technique called **Sobel Filtering**.

The code performs the following steps for every image:

1. **Grayscale Conversion:** It removes color because edges are defined by changes in brightness (intensity), not hue.
2. **Kernel Application:** It defines two matrices (kernels) that look for intensity changes:
    - **Sobel X:** Detects vertical lines.
    - **Sobel Y:** Detects horizontal lines.
3. **Convolution:** Using torch.nn.functional.conv2d, it slides these kernels across the image. Where there is a sharp contrast (like the edge of a face against a background), the math produces a high value.
4. **Magnitude Calculation:** It combines the X and Y results using the Pythagorean theorem:

$$\text{edges} = \sqrt{\text{edge}_x^2 + \text{edge}_y^2}$$

5. **Normalization:** It scales the values so the brightest edge is "1" (white) and the flat areas are "0" (black), ensuring the final image has proper contrast.

---

3. The Process Loop

The script iterates through every file in your source directory. To prevent a single corrupted image from crashing the whole script, it uses a try/except block—it will simply skip an image if it fails to open or process.

Performance Tracking

Because processing thousands of images can take time, the script includes a progress tracker that calculates:

- **Speed:** How many images are processed per second.
- **ETA:** A real-time estimate of how many minutes are left based on current performance.

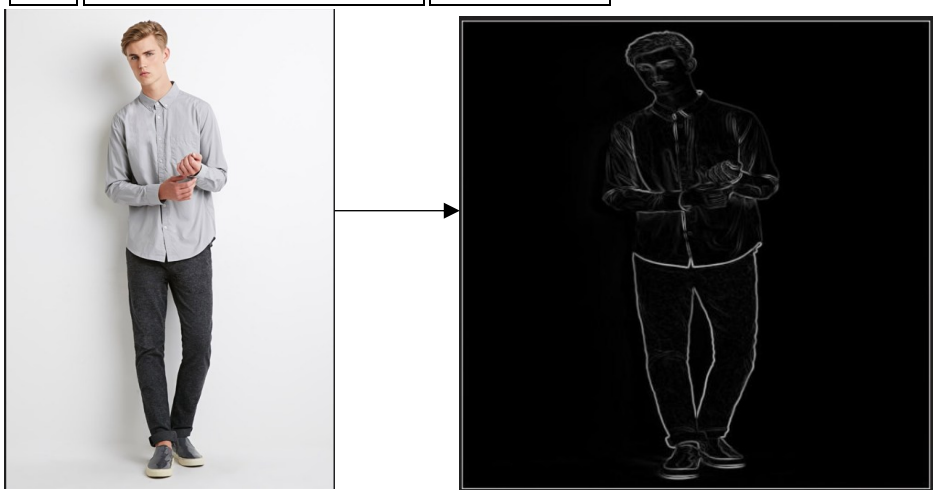
---

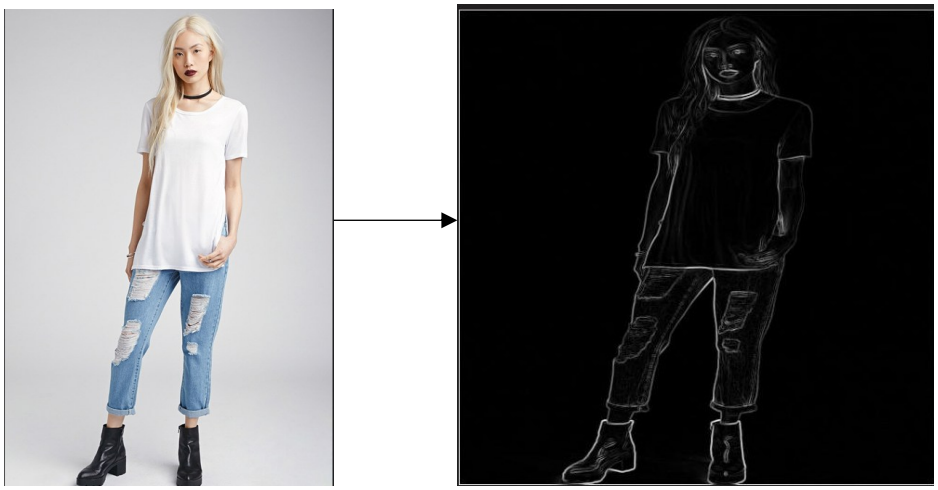
4. Output

Finally, the script moves the data back from the GPU to the CPU, converts the matrix back into a standard image format (PIL), and saves it to your Google Drive.

Summary of Data Flow

Step	Action	Location
1	Load Image from Drive	CPU / RAM
2	Resize & Convert to Tensor	CPU
3	Move to GPU	**GPU Memory**
4	Apply Sobel Convolution	**GPU Memory**
5	Move back to CPU	CPU / RAM
6	Save as .jpg/.png	Google Drive





Step 2: fine tuning the controlnet model

Code

1. High-Level Architecture

The code uses a "Frozen" Stable Diffusion v1.5 base and trains a **ControlNet** extension.

- **VAE (Variational Autoencoder):** Compresses images into a smaller "latent space" to save memory.
- **CLIP Text Encoder:** Converts the text prompt ("fashion clothing") into math vectors the model understands.
- **UNet:** The main "engine" that predicts and removes noise to create an image.

- **ControlNet:** The only part being trained. It looks at the sketch and tells the UNet where the lines of the clothing should be.

---

2. The Process Flow

Phase A: Data Preparation

1. **SketchDataset Class:** Loads pairs of images—the original photo and its corresponding sketch.
2. **Normalization:** Images are scaled to 256×256 pixels and normalized to a range of $[-1, 1]$, which is the standard input format for Stable Diffusion.
3. **Splitting:** 80% of the data is used for training, and 20% is held back for testing (evaluation).

Phase B: The Diffusion Training Logic

This script follows the **forward diffusion process**:

1. **Latent Encoding:** The real image is turned into a latent (compressed) representation by the VAE.
2. **Adding Noise:** A random "noise" pattern is added to that latent. The amount of noise is determined by a random timestep.
3. **The Task:** The model is shown the "noisy" image + the "sketch." It must predict exactly what the noise looks like so it can be subtracted to reveal the clean image.

Phase C: Optimization & Efficiency

Training heavy models like this requires massive memory. The script uses three specific tricks to fit this on a single GPU:

- **Gradient Checkpointing:** Saves memory by recalculating some parts of the model during the backward pass instead of storing everything.
- **AMP (Automatic Mixed Precision):** Performs math using 16-bit floats (half-precision) where possible to speed up calculations and save VRAM.
- **Freezing:** vae, text_encoder, and unet are locked (requires_grad_(False)). Only the controlnet parameters are updated.

---

3. Evaluation Metrics

After every epoch, the script runs the evaluate() function to see how well it's doing using two metrics:

- **MSE (Mean Squared Error):** Measures how close the predicted noise was to the actual noise.
- **SSIM (Structural Similarity Index):** Measures how visually similar the generated structure is to the ground truth. High SSIM means the model is following the sketch lines accurately.

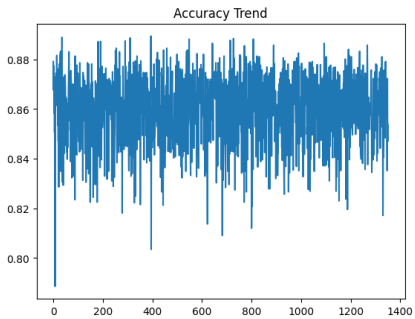
---

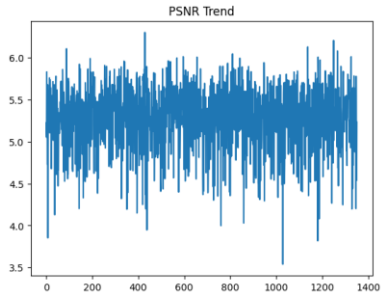
4. Summary of Operations

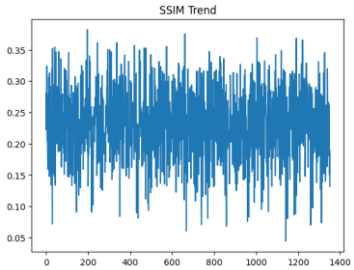
Component	Status	Role
VAE	Frozen	Compresses/Decompresses images.
UNet	Frozen	Provides the "creative" image generation power.
ControlNet	**Training**	Learns the relationship between sketches and photos.
Optimizer	AdamW	Updates the ControlNet weights based on the loss.
Loss	MSE	The "score" the model tries to improve

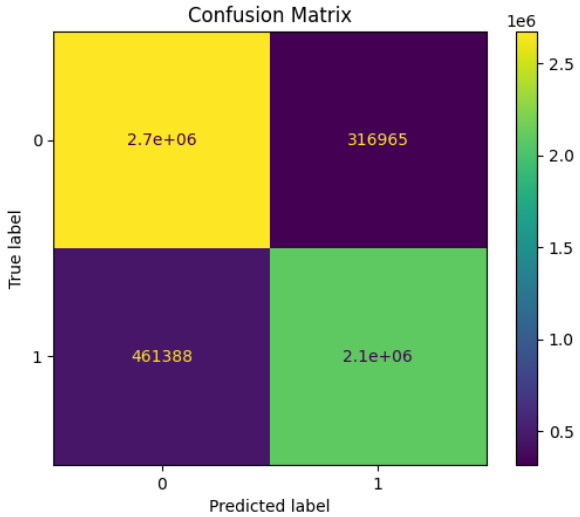
```
Epoch 0 | Step 4400 | Loss: 0.1455
Epoch 0 | Step 4500 | Loss: 0.2148
Epoch 0 | Step 4600 | Loss: 0.1585
Epoch 0 | Step 4700 | Loss: 0.1568
Epoch 0 | Step 4800 | Loss: 0.1578
Epoch 0 | Step 4900 | Loss: 0.1850
Epoch 0 | Step 5000 | Loss: 0.1770
Epoch 0 | Step 5100 | Loss: 0.1428
Epoch 0 | Step 5200 | Loss: 0.1949
Epoch 0 | Step 5300 | Loss: 0.1264
Epoch 0 | Step 5400 | Loss: 0.1726

✅ Epoch 0
Train Loss: 0.2615
Test Loss: 0.1403
SSIM: 0.2253
🎉 Training Complete!
```









Results

1. **Using the fine tuned model**
   Code

   1. Loading the "Brain" (Model Weights)
   The script doesn't just use a generic model; it specifically loads the custom-trained weights you created in the previous step.

- **Custom Weights:** controlnet.load_state_dict takes the .pth file from your Google Drive (the result of your hard work training) and injects that knowledge back into the ControlNet architecture.
- **Optimization:** It uses .half() and torch.float16. This converts the math from 32-bit to 16-bit, which makes the image generation **2x faster** and uses significantly less GPU memory without a noticeable loss in quality.

---

   2. Setting up the Pipeline
   The StableDiffusionControlNetPipeline acts as the "Manager." It coordinates three separate entities:

1. **The Prompt:** Your text description ("add the best fashion related to girl cloths").
2. **The Sketch:** The visual structure you uploaded.
3. **The Base Model:** Stable Diffusion v1.5, which provides the general knowledge of what "clothing" and "girls" look like.

---

   3. The Generation Process (Inference)
   When you run pipe(...), the following sequence occurs:

- **Image Processing:** Your uploaded sketch is resized to 512×512 to match the model's expected input size.
- **Attention Slicing:** The command pipe.enable_attention_slicing() breaks the massive math computations into smaller "slices." This is a lifesaver for Colab users, as it prevents "Out of Memory" (OOM) errors.
- **Diffusing the Image:**
  - The model starts with a canvas of pure random noise.
  - Over **20 steps** (num_inference_steps), the UNet looks at your prompt and your sketch.
  - The **ControlNet** acts as a "tracing paper," forcing the UNet to only place colors and textures within the lines of your sketch.
  - By step 20, the noise has been refined into a clear, high-resolution image.

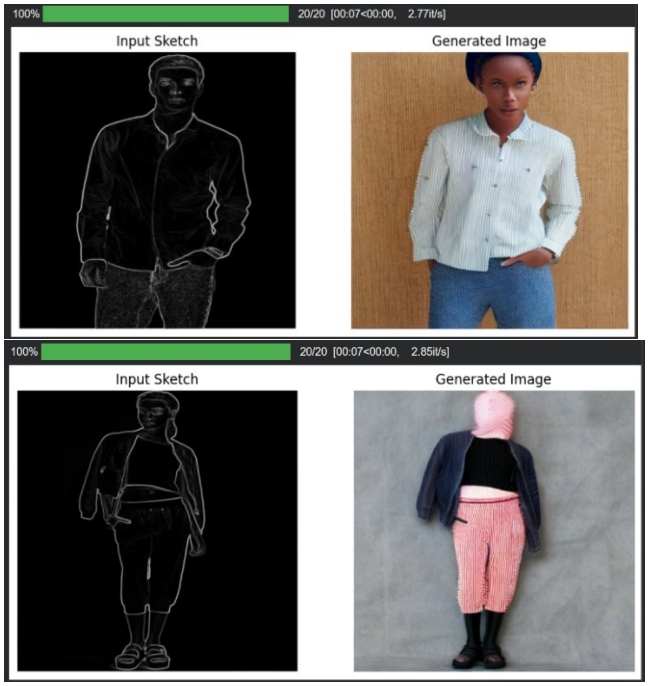
---

   4. Visualizing the Result
   The final section uses matplotlib to display a side-by-side comparison:

1. **Left:** The "Input Sketch" you provided.
2. **Right:** The "Generated Image" the AI created by "filling in" your sketch with realistic textures and lighting.

   Summary of Key Commands

Command	What it does
files.upload()	Lets you pick a file directly from your computer into the Colab environment.
torch.autocast("cuda")	Automatically chooses the best mathematical precision for your GPU to speed up generation.
num_inference_steps=20	Balance between speed and quality. Higher numbers (e.g., 50) take longer but add more detail.



2. **Without fine tuning controlenet**
   Code

   1. The Core Architecture
   This code uses a **ControlNet Pipeline**. Think of it as a three-part harmony:
1. **Stable Diffusion (The Artist):** Knows how to draw textures, lighting, and general "fashion."
2. **ControlNet (The Guide):** Specifically looks at your sketch to ensure the "Artist" stays inside the lines.
3. **The Prompt (The Direction):** Tells the model what specific style or item to create (e.g., "fashion related cloths").

---

   2. Key Technical Processes
   Loading the Weights

- **sd-controlnet-canny**: This specific model is designed to recognize edges (lines). Even though you are feeding it a sketch, the "Canny" model is excellent at interpreting those lines as structural boundaries.
- **.half() and torch.float16**: This is a memory optimization. It uses "Half Precision" math, which cuts the GPU VRAM usage in half and makes generation significantly faster without losing quality.

Pipeline Optimizations

- **enable_attention_slicing()**: This is a crucial trick for Google Colab. It breaks the large mathematical matrices into smaller chunks ("slices"). It prevents the common "CUDA Out of Memory" error.

---

3. The Generation Workflow

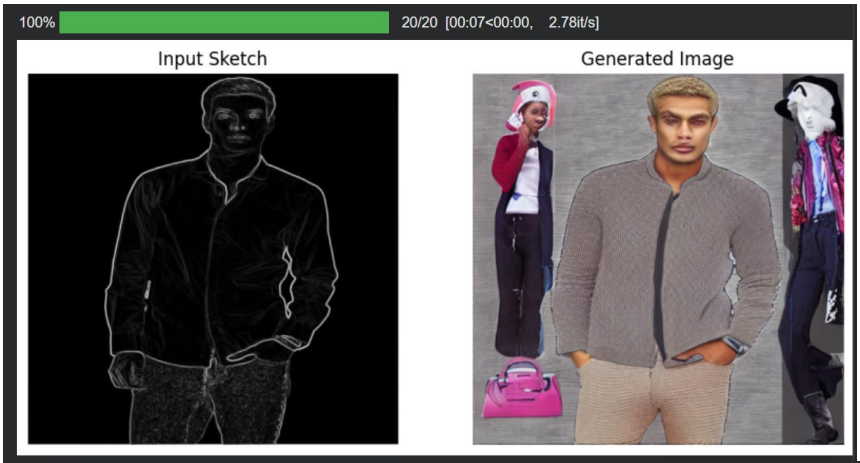
When you call pipe(...), the machine follows these steps:

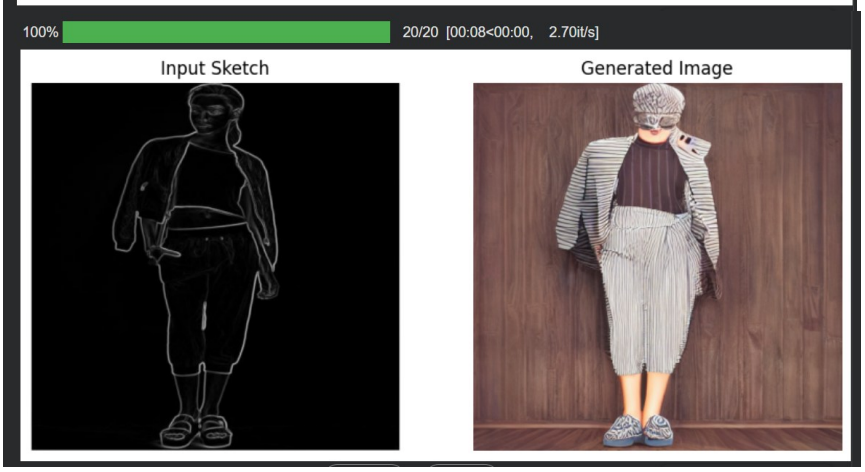
1. **Latent Initialization:** It starts with a canvas of pure random Gaussian noise.
2. **Conditioning:** It "looks" at your uploaded sketch. The ControlNet creates a feature map that tells the UNet, "Whatever you draw, the edges must match these lines."
3. **Iterative Denoising:** Over **20 steps** (num_inference_steps), the model slowly removes the noise.
    - In the first 5 steps, it decides the general colors and shapes.
    - In the last 5 steps, it adds fine details like fabric texture and lighting.
4. **VAE Decoding:** The final mathematical representation is converted back into a pixel-based image (.images[0]) that you can actually see.

---

4. Summary of Code Variables

Variable	Purpose
image	Your uploaded sketch, resized to 512×512 (the standard size for SD v1.5).
prompt	The text instruction that guides the "vibe" and "content" of the clothing.
num_inference_steps	Set to 20. Increasing this to 30 or 50 can improve quality but takes more time.
autocast("cuda")	Ensures the GPU uses the fastest possible math operations for the specific hardware.


100%
20/20 [00:07<00:00, 2.78it/s]
Input Sketch
Generated Image


100%
20/20 [00:08<00:00, 2.70it/s]
Input Sketch
Generated Image